LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

Modul 5

Dosen Pengampu:

1. Dr. Ir. Septafiansyah Dwi Putra - NIP : 198809112009121001
2. Agus Ambarwari, S.Pd., M.Kom. - NIP : 199208082019031020



Disusun oleh :

Ananda Haris Rahmadi (25782032)

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

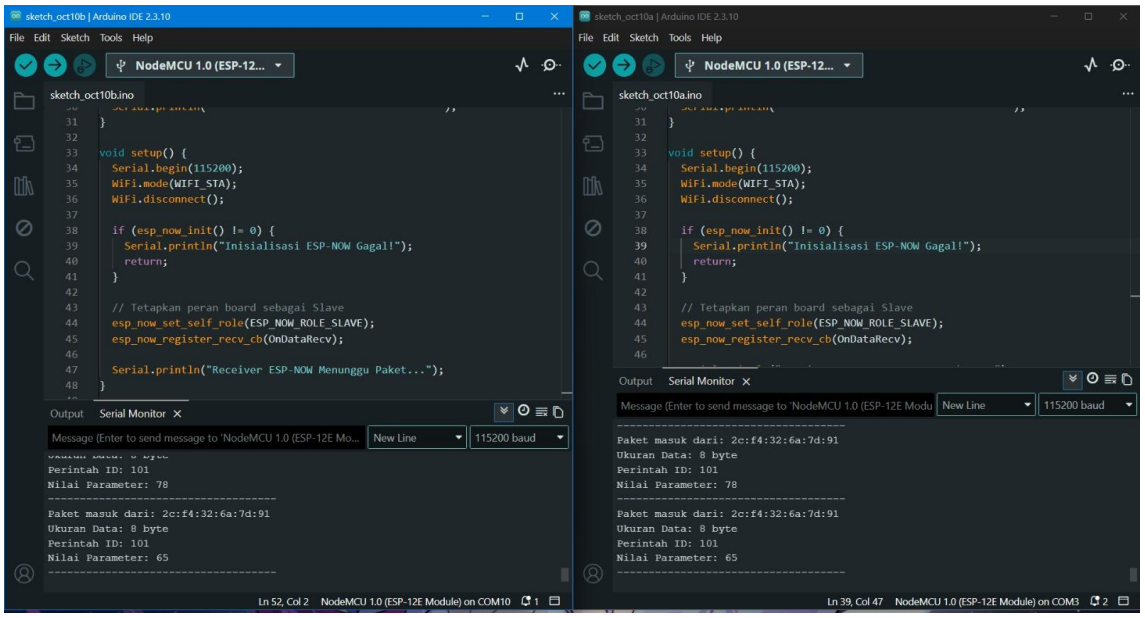
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

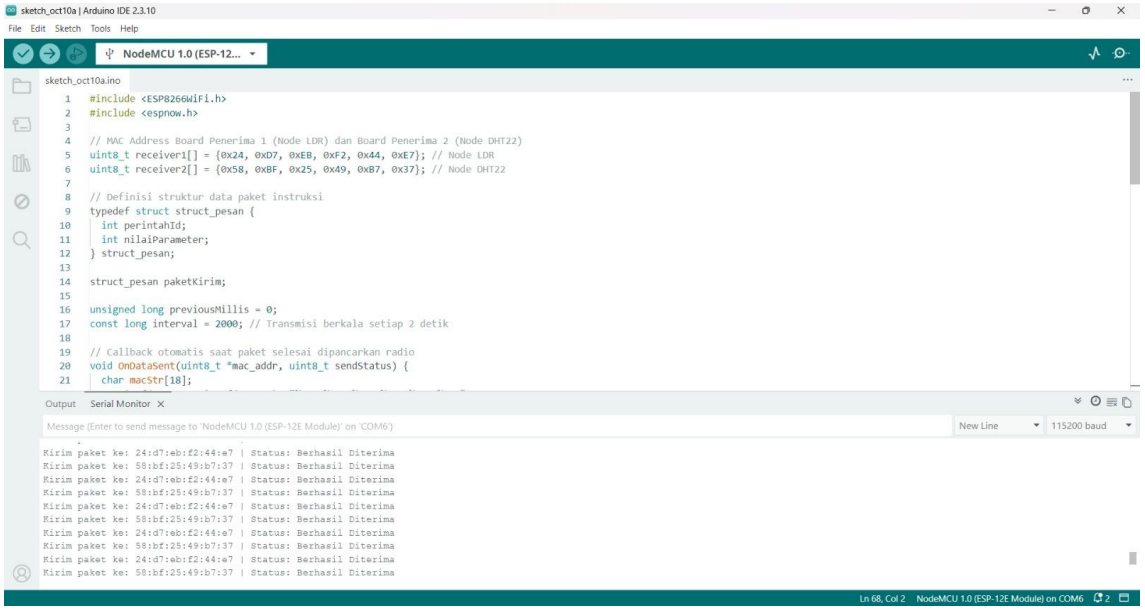
Praktikum2

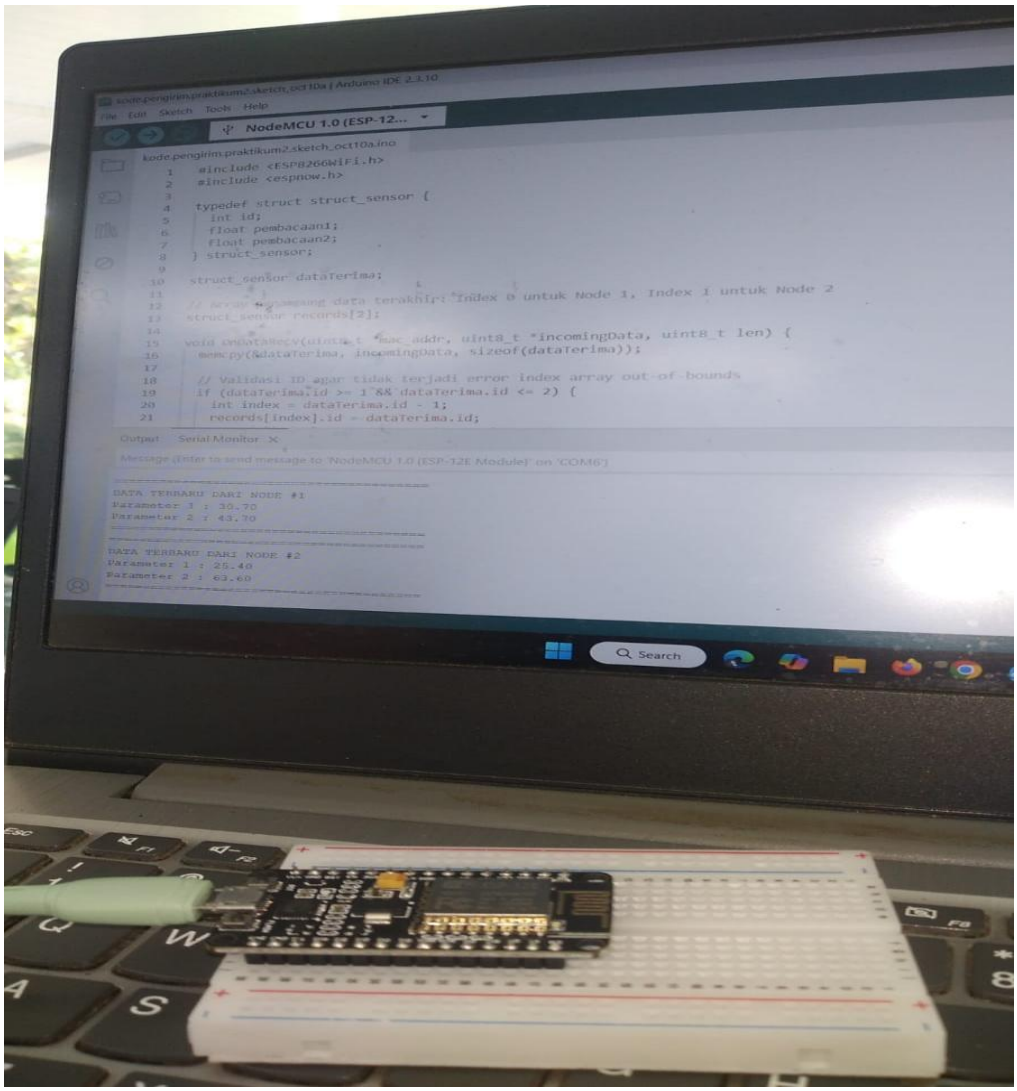
Sisi Penerima (Slave):





Sisi Pengirim (Controller):





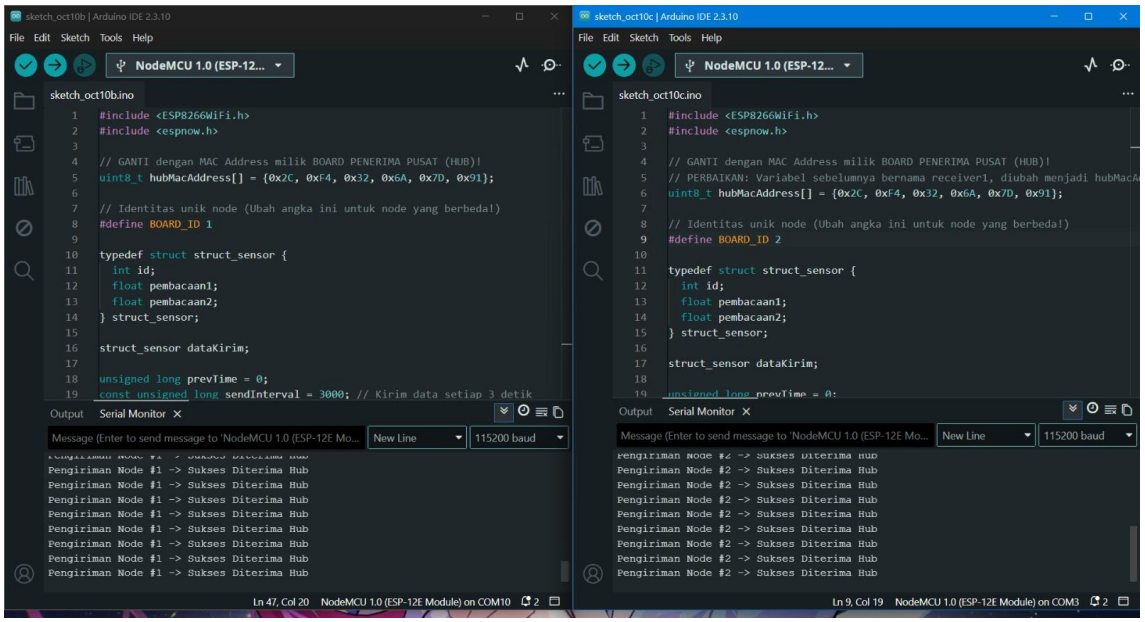
Dalam praktikum ini, saya mempelajari cara melakukan komunikasi nirkabel langsung antarpengontrol ESP8266 menggunakan protokol ESP-NOW dengan skema One-to-Many (satu pengirim ke banyak penerima) tanpa memerlukan perantara router Wi-Fi.

Poin-poin utama dari implementasi yang saya lakukan:

- Sisi Pengirim (Controller):
    - Saya mendaftarkan alamat MAC dari dua board penerima (receiver1 dan receiver2) ke dalam variabel array heksadesimal berawalan 0x.
    - Menggunakan fungsi esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER) untuk menetapkan peran board sebagai pengirim.
    - Memanfaatkan fungsi esp_now_add_peer() untuk mendaftarkan kedua board penerima ke tabel peer internal.
    - Mengirimkan paket data berisi perintahId (101) dan nilaiParameter acak secara berkala setiap 2 detik menggunakan fungsi esp_now_send(0, ...) di mana argumen 0 berarti paket disiarkan ke seluruh peer yang terdaftar.
    - Dilengkapi fungsi callback OnDataSent untuk memantau status pengiriman apakah berhasil diterima atau gagal.
- Sisi Penerima (Slave):
    - Board penerima dikonfigurasi menggunakan peran ESP_NOW_ROLE_SLAVE dan tidak perlu mengetahui alamat MAC si pengirim.
    - Menggunakan fungsi callback interupsi OnDataRecv yang otomatis aktif saat antena mendeteksi paket masuk.
    - Menerapkan perintah memcpy(&paketTerima, incomingData, sizeof(paketTerima)) untuk menyalin data biner mentah dari buffer memori radio ke dalam struktur data variabel agar bisa dibaca di Serial Monitor.

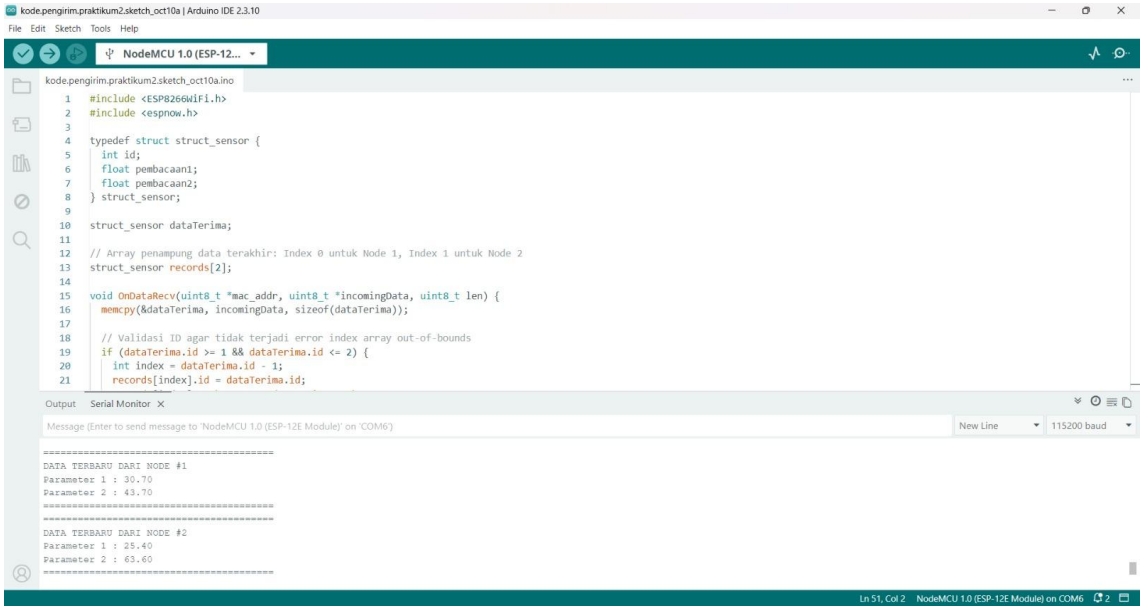
Praktikum3

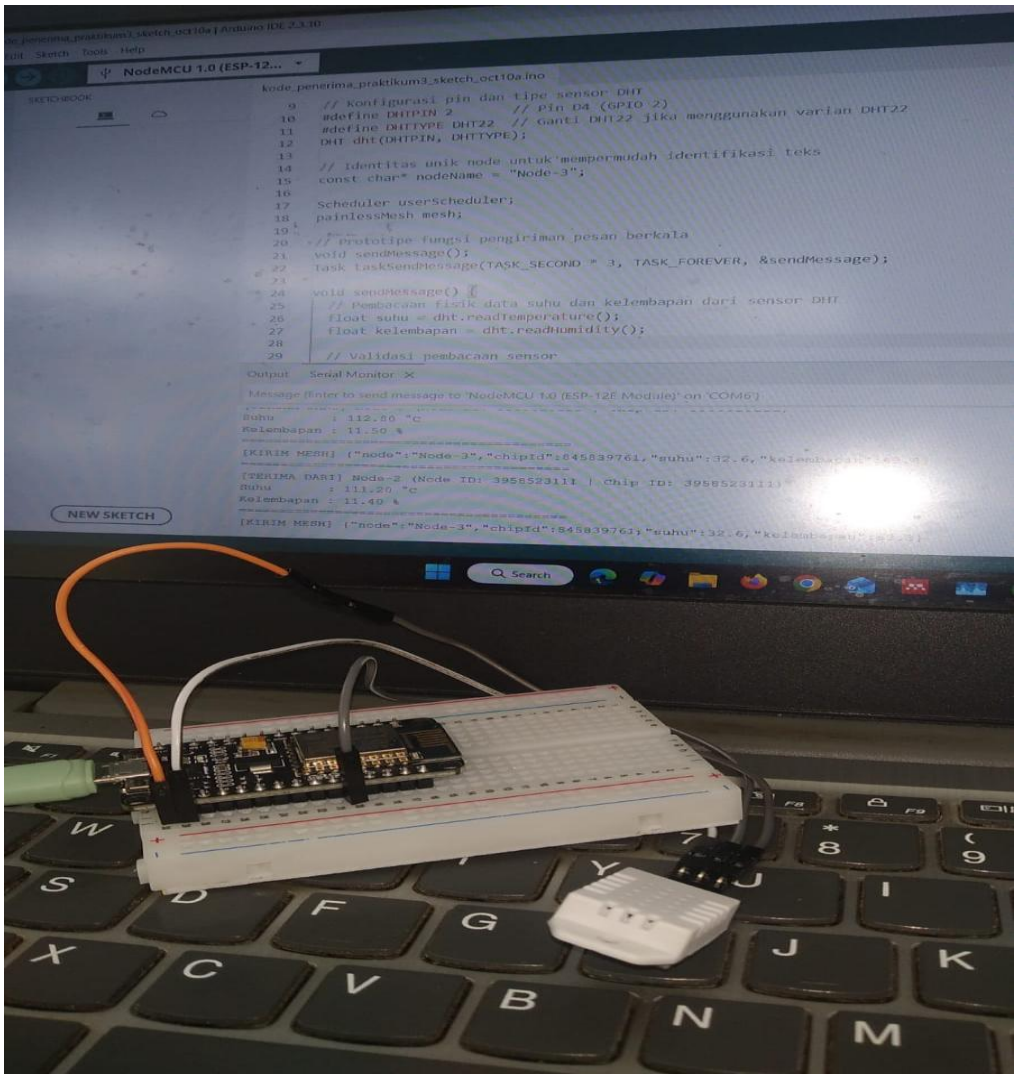
Pengirim 1 dan 2





Penerima


kode.pengirim.praktikum2.sketch_oct10a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
kode.pengirim.praktikum2.sketch_oct10a.ino
#include <ESP8266WiFi.h>
#include <espnow.h>
typedef struct struct_sensor {
int id;
float pembacaan1;
float pembacaan2;
} struct_sensor;
struct_sensor dataTerima;
// Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
struct_sensor records[2];
void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
memcpy(&dataTerima, incomingData, sizeof(dataTerima));
// Validasi ID agar tidak terjadi error index array out-of-bounds
if (dataTerima.id >= 1 && dataTerima.id <= 2) {
int index = dataTerima.id - 1;
records[index].id = dataTerima.id;
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')
New Line
115200 baud
DATA TERBARU DARI NODE #1
Parameter 1 : 30.70
Parameter 2 : 43.70
DATA TERBARU DARI NODE #2
Parameter 1 : 25.40
Parameter 2 : 63.60
Ln 51, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM6

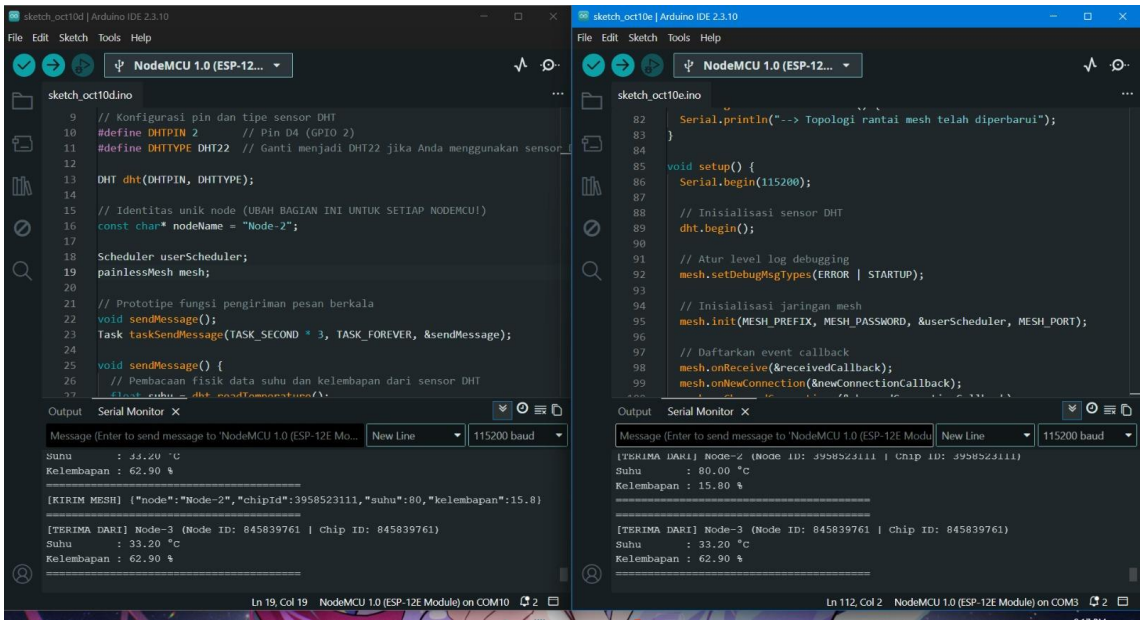

NodeMCU 1.0 (ESP-12...
kode_penerima_praktikum3_sketch_oct10a.ino
// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN 2 // Pin D4 (GPIO 2)
#define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
DHT dht(DHTPIN, DHTTYPE);
// Identitas unik node untuk mempermudah identifikasi teks
const char* nodeName = "Node-3";
Scheduler userScheduler;
painlessMesh mesh;
// Prototipe fungsi pengiriman pesan berkala
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
void sendMessage() {
// Pembacaan fisik data suhu dan kelembapan dari sensor DHT
float suhu = dht.readTemperature();
float kelembapan = dht.readHumidity();
// Validasi pembacaan sensor
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')
Suhu : 112.80 °C
Kelembapan : 11.50 %
[KIRIM MESH] {"node":"Node-3","chipId":845839761,"suhu":32.6,
[TERIMA DARI] Node-2 (Node ID: 3958523111 | Chip ID: 3958523111)
Suhu : 111.20 °C
Kelembapan : 11.40 %
[KIRIM MESH] {"node":"Node-3","chipId":845839761,"suhu":32.6,
NEW SKETCH
Search

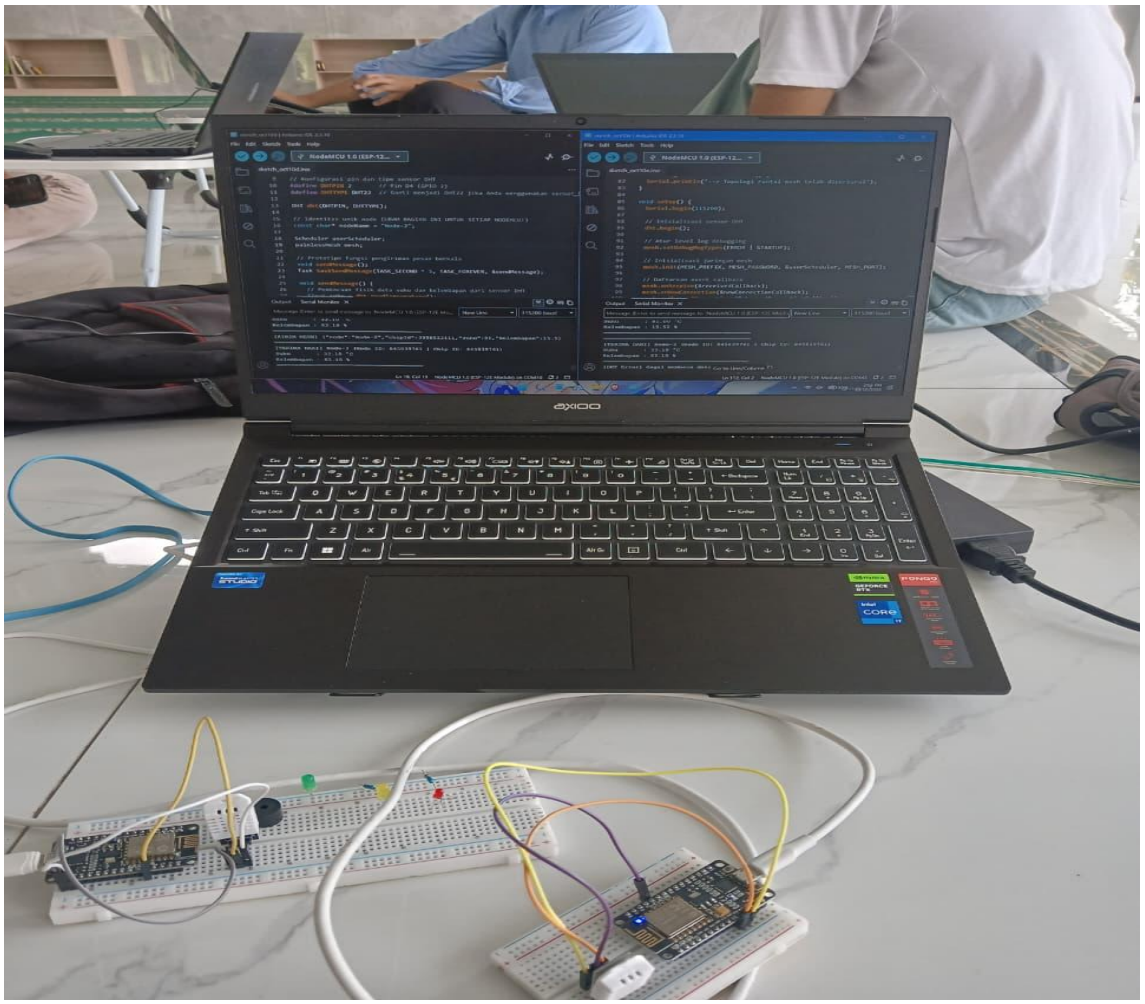
Dalam praktikum ini, saya mempelajari cara melakukan pengumpulan data telemetri menggunakan arsitektur Many-to-One (banyak node pengirim ke satu node pusat/hub) via protokol ESP-NOW.

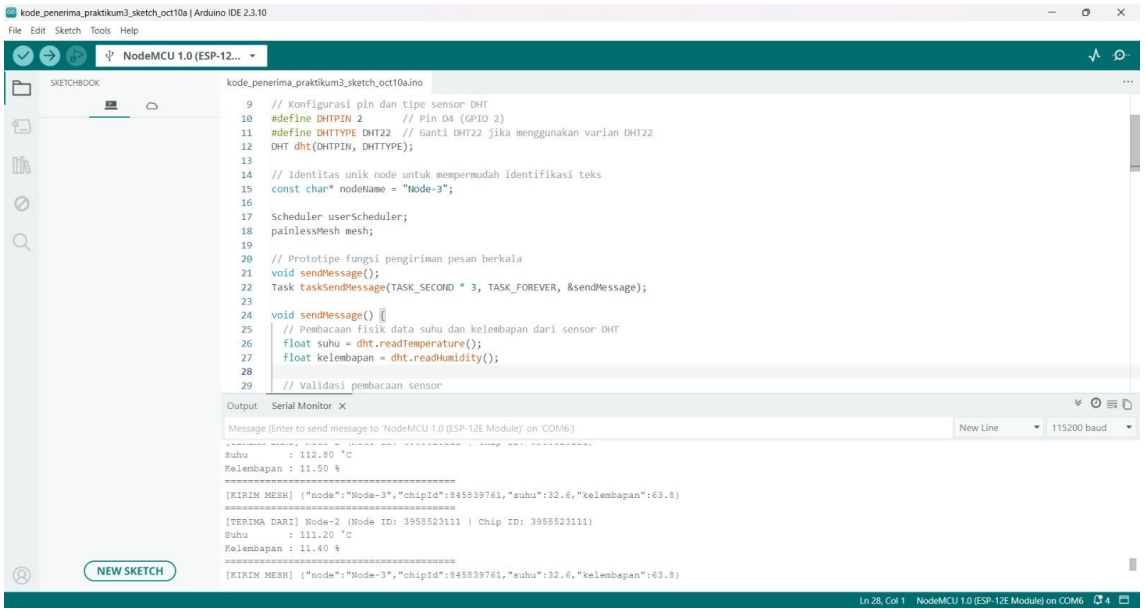
Poin-poin utama dari implementasi yang saya buat:

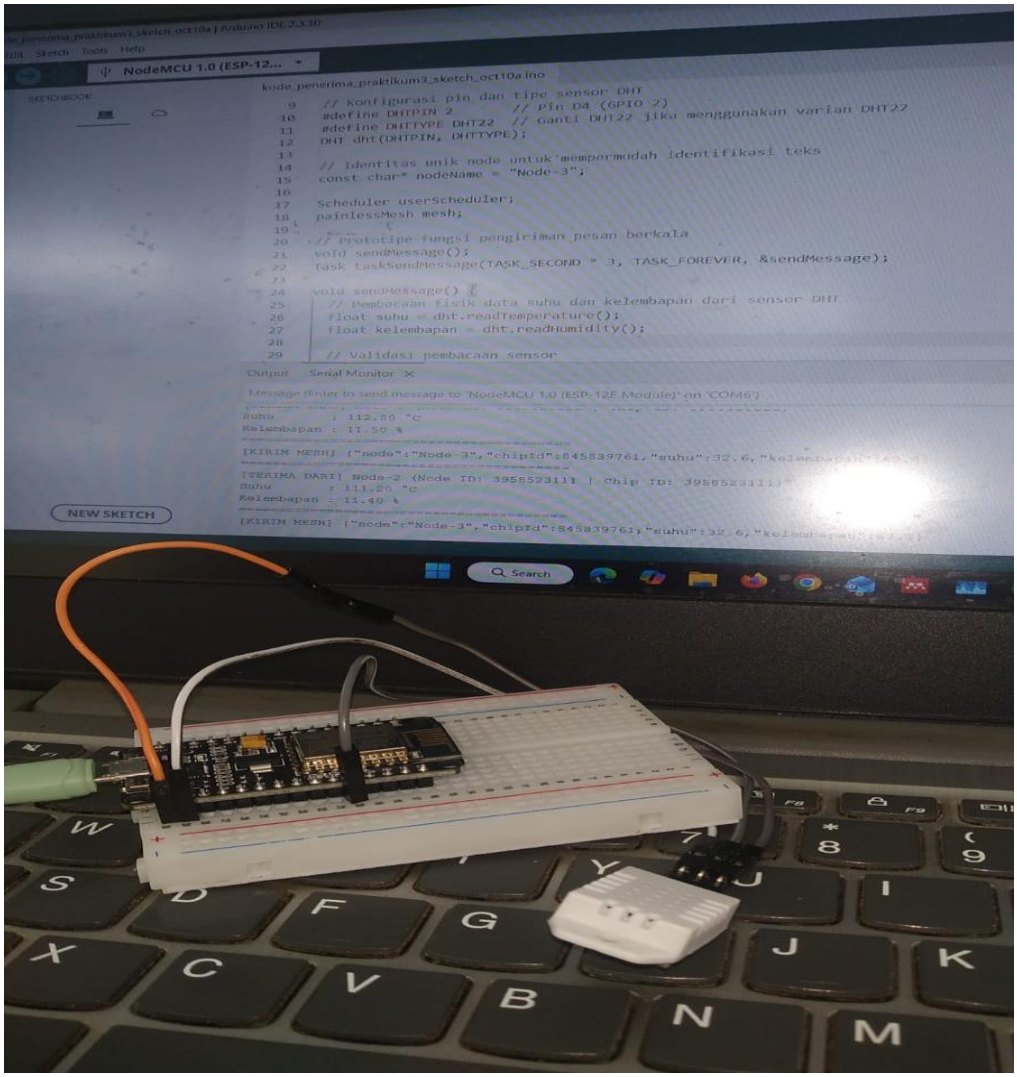
- Sisi Node Pengirim (Sensor Node):
    - Saya menyiapkan dua node pengirim yang dibedakan menggunakan makro identitas unik #define BOARD_ID (diatur angka 1 untuk node pertama dan 2 untuk node kedua).
    - Menyiapkan variabel array hubMacAddress untuk mendaftarkan alamat MAC milik board penerima pusat (Hub).
    - Mengirimkan paket data simulasi sensor secara berkala setiap 3 detik menggunakan struktur data struct_sensor yang mencakup ID node, serta dua parameter pembacaan (pembacaan1 untuk suhu dan pembacaan2 untuk kelembapan).
    - Menggunakan fungsi esp_now_send() yang diarahkan langsung ke alamat MAC Hub, disertai callback OnDataSent untuk memantau status keberhasilan pengiriman paket dari masing-masing node.
- Sisi Hub Konsentrator (Base Station):
    - Board penerima pusat dikonfigurasi sebagai slave (ESP_NOW_ROLE_SLAVE) untuk bersiap menerima transmisi data masuk dari berbagai node lapangan tanpa perlu mendata alamat MAC pengirim satu persatu.
    - Menggunakan fungsi callback interupsi OnDataRecv yang otomatis aktif saat paket data diterima, lalu menyalin buffer memori menggunakan memcpy().
    - Menerapkan validasi ID (dataTerima.id >= 1 && dataTerima.id <= 2) untuk mencegah error out-of-bounds, lalu menyimpan datanya ke dalam array penyimpanan records[] berdasarkan indeks yang sesuai untuk ditampilkan ke Serial Monitor.

Praktikum 4


sketch_oct10d | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct10d.ino
// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN 2 // Pin D4 (GPIO 2)
#define DHTTYPE DHT22 // Ganti menjadi DHT22 jika Anda menggunakan sensor
DHT dht(DHTPIN, DHTTYPE);
// Identitas unik node (UBAH BAGIAN INI UNTUK SETIAP NODEMCU!)
const char* nodeName = "Node-2";
Scheduler userScheduler;
painlessMesh mesh;
// Prototipe fungsi pengiriman pesan berkala
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
void sendMessage() {
// Pembacaan fisik data suhu dan kelembapan dari sensor DHT
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Mo...
New Line
115200 baud
Kelembapan : 62.90 %
[KIRIM MESH] {"node":"Node-2","chipId":3958523111,"suhu":80,"kelembapan":15.8}
[TERIMA DARI] Node-3 (Node ID: 845839761 | Chip ID: 845839761)
Suhu : 33.20 °C
Kelembapan : 62.90 %
Ln 19, Col 19 NodeMCU 1.0 (ESP-12E Module) on COM10
sketch_oct10e | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct10e.ino
Serial.println("--> Topologi rantai mesh telah diperbarui");
}
void setup() {
Serial.begin(115200);
// Inisialisasi sensor DHT
dht.begin();
// Atur level log debugging
mesh.setDebugMsgTypes(ERROR | STARTUP);
// Inisialisasi jaringan mesh
mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
// Daftarkan event callback
mesh.onReceive(&receivedCallback);
mesh.onNewConnection(&newConnectionCallback);
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Modu
New Line
115200 baud
Suhu : 80.00 °C
Kelembapan : 15.80 %
[TERIMA DARI] Node-3 (Node ID: 845839761 | Chip ID: 845839761)
Suhu : 33.20 °C
Kelembapan : 62.90 %
Ln 112, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3




kode_penerima_praktikum3_sketch_oct10a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
SKETCHBOOK
kode_penerima_praktikum3_sketch_oct10a.ino
// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN 2 // Pin D4 (GPIO 2)
#define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
DHT dht(DHTPIN, DHTTYPE);
// Identitas unik node untuk mempermudah identifikasi teks
const char* nodeName = "Node-3";
Scheduler userScheduler;
painlessMesh mesh;
// Prototipe fungsi pengiriman pesan berkala
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
void sendMessage() {
// Pembacaan fisik data suhu dan kelembapan dari sensor DHT
float suhu = dht.readTemperature();
float kelembapan = dht.readHumidity();
// Validasi pembacaan sensor
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')
New Line
115200 baud
Suhu : 112.80 °C
Kelembapan : 11.50 %
[KIRIM MESH] {"node":"Node-3","chipId":845839761,"suhu":32.6,"kelembapan":63.8}
[TERIMA DARI] Node-2 (Node ID: 3958523111 | Chip ID: 3958523111)
Suhu : 111.20 °C
Kelembapan : 11.40 %
[KIRIM MESH] {"node":"Node-3","chipId":845839761,"suhu":32.6,"kelembapan":63.8}
NEW SKETCH
Ln 28, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM6


NodeMCU 1.0 (ESP-12...
NEW SKETCH
Search

Dalam praktikum ini, saya mempelajari cara membangun jaringan komunikasi mesh otonom (multi-hop) tanpa memerlukan router pusat, di mana setiap node dapat saling berbagi data telemetri sensor suhu dan kelembapan secara langsung.

Poin-poin utama dari implementasi yang saya buat:

- Perancangan Hardware & Konfigurasi Identitas:
    - Sensor DHT (DHT11/DHT22) dihubungkan ke setiap board NodeMCU dengan konfigurasi pin VCC ke 3V3, GND ke GND, dan pin data/OUT ke pin D4 (GPIO 2).
    - Setiap board diberikan nama pengenal unik melalui variabel nodeName (misalnya "Node-1", "Node-2", dst.).
- Format Data JSON & Pengiriman Broadcast:
    - Setiap node secara berkala membaca nilai suhu dan kelembapan dari sensor fisiknya menggunakan fungsi pustaka DHT.
    - Data tersebut divalidasi, lalu dikemas ke dalam format objek teks JSON menggunakan pustaka ArduinoJson yang mencakup nama node, chip ID, suhu, dan kelembapan.
    - Paket pesan JSON tersebut disiarkan ke seluruh jaringan mesh menggunakan fungsi mesh.sendBroadcast(msg) agar dapat diterima oleh node lain secara multi-hop.
- Manajemen Jaringan & Mekanisme Callback:
    - Menggunakan pustaka painlessMesh untuk menangani pembentukan jaringan soft-AP dan stasiun secara simultan di latar belakang.
    - Menerapkan fungsi callback penerimaan receivedCallback() untuk membedah (deserialize) string JSON yang masuk kembali menjadi variabel terpisah agar bisa ditampilkan ke Serial Monitor.
    - Memanfaatkan TaskScheduler (userScheduler) dan fungsi pemeliharaan mesh.update() di dalam loop untuk menggantikan fungsi delay(), sehingga proses perutean paket (routing) latar belakang tidak terganggu dan topologi mesh tetap stabil.